

Laboratory work 1

1 Objectives

The objective of this laboratory is to describe briefly the SDL library and to exemplify the development of a basic SDL based application.

2 Create a window using SDL

SDL (Simple DirectMedia Layer) library manages the access to graphics hardware (via OpenGL and Direct3D libraries) and also to audio, keyboard and mouse (independent on the underlying operating system). It supports Windows, Mac OS X and Linux and various other platforms. During this laboratory we will be using the C++ programming language but there are available bindings to other languages (such as C# or Python).

In order to create an SDL window the following steps are required:

1. Initialize the SDL library by calling the [SDL_Init\(\)](#) function with `SDL_INIT_VIDEO` as argument because we are using only the video subsystem of the SDL.

```
SDL_Init(SDL_INIT_VIDEO);
```

2. Create the window and renderer using the [SDL_CreateWindowAndRenderer\(\)](#) function. The arguments are the window title, width, height, some flags (for example to create a fullscreen window or a resizable window) and the objects that will store references to the created window and renderer.

```
SDL_CreateWindowAndRenderer("SDL Hello World Example", WINDOW_WIDTH,  
WINDOW_HEIGHT, SDL_WINDOW_HIGH_PIXEL_DENSITY, &window, &renderer);
```

Before closing the application, we need to deallocate all the resources we created:

1. Destroy the renderer by calling the [SDL_DestroyRenderer\(\)](#) function and passing as the argument the pointer to the renderer.

```
SDL_DestroyRenderer(renderer);
```

2. Destroy the window by calling the [SDL_DestroyWindow\(\)](#) function and passing as the argument the pointer to the window.

```
SDL_DestroyWindow(window);
```

3. Call the [SDL_Quit\(\)](#) function that is responsible to clean up all initialized subsystems (for our example we are using only the video subsystem).

```
SDL_Quit();
```

3 SDL Renderer

SDL_Renderer is a structure that handles all rendering. It is tied to an **SDL_Window** so it can only render within that **SDL_Window**. It also keeps track of the settings related to the rendering.

There are several important functions tied to the **SDL_Renderer**:

1. **SDL_SetRenderDrawColor** sets the color that will be used in all drawing operations, until another call to the function is performed.

```
SDL_SetRenderDrawColor(renderer, r, g, b, a);
```

2. **SDL_RenderClear** clears the entire window display area using the current active color, previously set with **SDL_SetRenderDrawColor** function.

```
SDL_RenderClear(renderer);
```

3. **SDL_RenderPresent** will display everything drawn in the renderer to the screen. Until this function is called, all the drawing takes place in a hidden buffer that is not visible to the user. The call to **SDL_RenderPresent** should be made only once, after all the drawing functions have been called.

```
SDL_RenderPresent(renderer);
```

If we want to draw a rectangle, we need to specify the starting position of the rectangle (the x and y coordinates) and the dimension (width and height). In order to specify the color we are using the **SDL_Color** structure using the Red, Green, Blue and Alpha values. The **SDL_RenderFillRect()** function will draw the rectangle with the previously specified color.

```
SDL_FRect rectangleCoordinates = {100, 100, 200, 200};
SDL_Color rectangleColor { 255, 0, 0, 255 };
SDL_SetRenderDrawColor(renderer,
    rectangleColor.r, rectangleColor.g, rectangleColor.b, rectangleColor.a);
SDL_RenderFillRect(renderer, &rectangleCoordinates);
```

For each color channel we specify the values between 0 and 255. Look at the following table for some color channel values, by combining the channel values we get different colors.

Color name	Red channel	Green channel	Blue channel	Color
White	255	255	255	
Red	255	0	0	
Green	0	255	0	
Blue	0	0	255	
Yellow	255	255	0	
Black	0	0	0	

4 Processing events

In SDL the events could be things like pressing a keyboard key or mouse motion. All the events are stored in a queue in the order in which they occurred. By using the [SDL_PollEvent\(\)](#) function we check for and process the next event from the queue.

4.1 Mouse pressed event

We can check if the left mouse button is pressed using the following piece of code, first we check the event type to be **SDL_EVENT_MOUSE_BUTTON_DOWN** and the pressed button to be **SDL_BUTTON_LEFT**. The **SDL_GetMouseState()** function gets the coordinates of the mouse current position (x and y).

```
if(currentEvent.type == SDL_EVENT_MOUSE_BUTTON_DOWN)
    if(currentEvent.button.button == SDL_BUTTON_LEFT)
        SDL_GetMouseState(&mouseX, &mouseY);
```

4.2 Mouse move event

We can check if the left mouse button is pressed while the mouse is moving using the following piece of code, first we check the event type to be **SDL_EVENT_MOUSE_MOTION** and the pressed button to be **SDL_BUTTON_LEFT**.

```
if(currentEvent.type == SDL_EVENT_MOUSE_MOTION)
{
    SDL_MouseButtonFlags mouseButtons = SDL_GetMouseState(nullptr, nullptr);
    if( mouseButtons & SDL_BUTTON_MASK(SDL_BUTTON_LEFT))
        SDL_GetMouseState(&mouseX, &mouseY);
}
```

4.3 Keyboard event

We can get the key pressed by using the following piece of code, first we check the event type to be **SDL_EVENT_KEY_DOWN** and then we process the desired keys.

```
if(currentEvent.type == SDL_EVENT_KEY_DOWN)
    switch(currentEvent.key.key)
    {
        case SDLK_UP:
            //process UP key
            break;

        case SDLK_R:
            //process R key
            break;
        ...
        default:
            //default process
            break;
    }
```

5 Further reading

- Setting up SDL3 on various platforms – <https://wiki.libsdl.org/SDL3/Tutorials/FrontPage>
- SDL3 Tutorials - <https://lazyfoo.net/tutorials/SDL3/index.php>

6 Assignment

Download and run the application from the laboratory website. Try to understand the basic example and then extend the application with the following functionality:

- Change the rectangle color by selecting the color channel using the R (for red), G (for green) and B (for blue) keys and by selecting the channel value by pressing the UP and DOWN keys.
- Interactively display a rectangle, the first set of coordinates is retrieved at left mouse button down event and the second set of coordinate is retrieved at every mouse move event (but only if the left button is still pressed).